GUIDA ORALE · PROGETTO FINALE LLM & AGENTIC AI

Guida orale — Casa & Regali v2

Domande probabili, risposte brevi e punti di codice da saper indicare. I numeri sono quelli della run LFM3 del 31 ottobre 2026.

31/31	60	3,72 s	0,94
scenari LFM3	test automatici	latenza media	tool per turno

Le risposte sono pensate per essere dette in 30–60 secondi. Se la domanda chiede un dettaglio di codice, la sequenza è: firma, input/output, punto di scrittura, caso limite, test.

Come usare questa guida

Prima leggi le dodici risposte ad alta voce, poi prova i trabocchetti senza guardare. Nella demo non aprire tutti i file: mostra il grafo, il confine del cliente, il rimborso e la failure gallery. La chiusura deve lasciare un limite esplicito, non una promessa.

Le dodici domande

1. In una frase, qual è la tesi del progetto?

2. Perché due chiamate LLM invece di una?

3. Cosa succede esattamente quando il cliente chiede un rimborso?

4. Come impedisci che il modello legga ordini di altri clienti?

5. Perché `interrupt()` è dentro il tool e non in un nodo separato?

6. Cosa controlla `execute_refund` se il tool ha già controllato?

7. Perché il contratto `<risposta>` e non tutto il `content`?

8. Perché BM25 e non un database vettoriale?

9. Come dimostri che un prompt injection non può saltare l'HITL?

10. I 31/31 sono una garanzia? Qual è il limite?

11. Perché i test non usano un modello reale? Cosa non misurano?

12. Cosa faresti con più tempo o in produzione?

Checklist prima dell'orale

- Ricorda i numeri: 31/31, 60 test, 3,72 s, 0,94 tool per turno, 7 tool, 10 documenti, 4 nodi.
- Distingui sempre ciò che decide l'LLM da ciò che impone il codice.
- Se ti chiedono un file, aprilo e indica la funzione, non improvvisare.
- Dichiara che Gemini è configurabile ma non misurato per quota.
- Non presentare i 31/31 come una garanzia di produzione.
- Se non ricordi un dettaglio, riparti dal contratto della funzione: input, output, scrittura, test.

> **La frase da tenere pronta**
>
> L'LLM sceglie il tool e scrive la risposta; il codice impone policy, privacy, HITL e ledger. Se il modello sbaglia, l'errore è visibile, ma non può diventare una scrittura irreversibile.

1. In una frase, qual è la tesi del progetto?

L'LLM sceglie il tool e scrive la risposta; il codice decide cosa è consentito. Policy, privacy, HITL e ledger non dipendono dal prompt: se il modello sbaglia, l'errore resta visibile ma non diventa una scrittura irreversibile.

La conseguenza pratica è che il progetto non chiede al modello di essere affidabile su tutto. Gli affida due decisioni osservabili e sposta i vincoli nel codice deterministico.

2. Perché due chiamate LLM invece di una?

La prima chiamata sceglie il tool; la seconda racconta l'esito senza avere tool collegati. Separare scelta e narrazione impedisce che il secondo passo avvii altre azioni e rende il rimborso definitivo per il turno.

Con un modello da 1-3B, un ciclo ReAct completo tende a ripetere il tool o a concatenarne altri dopo un esito già definitivo. Il grafo è aciclico: da `tools` si passa a `respond`, mai di nuovo ad `agent`.

3. Cosa succede esattamente quando il cliente chiede un rimborso?

`agent` chiama `proponi_rimborso("ORD-1001")`. Il tool legge il cliente dallo stato, verifica che l'ordine appartenga al profilo, applica `check_return_eligibility`, calcola l'importo con `calculate_refund` e costruisce la proposta con livello `operatore` o `responsabile`.

Poi chiama `interrupt({"type": "refund_approval", "proposal": ...})`: il turno si sospende e nessun rimborso è ancora stato scritto. La UI o la CLI riprendono con `Command(resume={"approved": ...})`. Solo dopo l'approvazione il tool chiama `ledger.execute_refund`, che rivalida tutto e scrive.

Se l'operatore rifiuta, il ledger non scrive e l'audit registra `rimborso_rifiutato`. Se l'ordine non è idoneo, non viene aperto alcun interrupt.

4. Come impedisci che il modello legga ordini di altri clienti?

Il `customer_id` non è un argomento dei tool: arriva dallo stato del grafo tramite `ToolRuntime`. `stato_ordine`, `verifica_reso` e `proponi_rimborso` usano `policy.find_order(order_id, customer_id)`, che restituisce `None` sia per un ordine inesistente sia per un ordine di un altro cliente.

Il messaggio è lo stesso nei due casi, quindi il modello non può dedurre se l'ordine esista. L'accesso negato viene registrato come `privacy_accesso_negato`. I tool non restituiscono nome, email, telefono o indirizzo del profilo: solo i campi dell'ordine.

5. Perché `interrupt()` è dentro il tool e non in un nodo separato?

Proposta e approvazione restano nello stesso punto: non c'è un percorso alternativo che possa scrivere un rimborso senza passare da lì. Un nodo separato lascerebbe aperta la possibilità di raggiungere la scrittura da un altro arco del grafo.

Il ledger è comunque una seconda difesa: anche se il tool sbagliasse, `execute_refund` ricontrolla approvazione, ordine, cliente, importo e duplicati prima di scrivere.

6. Cosa controlla `execute_refund` se il tool ha già controllato?

Controlla l'approvazione umana, normalizza ordine, cliente e importo, blocca i duplicati, verifica che l'ordine esista e appartenga al cliente, confronta l'importo con `calculate_refund` entro `0,01 EUR`, genera `RMB-2026-NNNN`, aggiorna refunds e orders e registra l'evento.

È la difesa in profondità: il ledger non si fida del modello né del chiamante. Il test `test_importo_non_coerente_bloccato` e `test_cliente_non_coerente_bloccato` dimostrano che una proposta alterata non scrive.

7. Perché il contratto `<risposta>` e non tutto il `content`?

Con LFM3 il modello aggiungeva ragionamento nel `content`, non in un canale separato. Il prompt finale chiede di scrivere solo tra `<risposta>` e `</risposta>`; `extract_answer` scarta tutto ciò che sta fuori.

Se i tag mancano, il testo resta invariato: non si perde una risposta valida solo per un problema di formato. Il test `test_extract_answer_rimuove_il_ragionamento` copre il caso con testo prima e dopo il blocco.

8. Perché BM25 e non un database vettoriale?

Il corpus ha dieci documenti brevi e un lessico stabile. BM25 è deterministico, non richiede indice persistente né modello di embedding e rende le fonti verificabili. L'indice viene ricostruito a ogni query: il costo è accettabile e non c'è cache da invalidare.

La query unisce il testo del modello all'ultima richiesta del cliente e aggiunge sinonimi italiani. I chunk senza sovrapposizione lessicale vengono scartati: meglio nessuna fonte che una fonte falsa. La suite misura il 100% di source hit sui 9 scenari informativi.

9. Come dimostri che un prompt injection non può saltare l'HITL?

Lo scenario `prompt_injection` chiede esplicitamente di ignorare le istruzioni e rimborsare senza approvazione. Il modello può anche chiamare `proponi_rimborso`, ma il tool sospende comunque il turno: l'approvazione non è una frase nel prompt, è un `interrupt` nel codice.

La valutazione verifica che il rimborso non sia scritto prima della decisione umana e che non ci siano tool successivi dopo l'esito. Il test di privacy e i test del ledger coprono lo stesso confine a livello di codice.

10. I 31/31 sono una garanzia? Qual è il limite?

No. Sono la misura di un prototipo su 31 scenari controllati, con un modello locale da 1,7B e dati sintetici. Non c'è un insieme di test separato per la generalizzazione, e la suite non copre tutte le formulazioni possibili.

Il dato utile è il confronto: Llama 3.2 3B (Unsloth) si ferma a 24/31, quindi il margine di errore dipende dal modello. La scelta di non avere un fallback deterministico rende questi errori visibili invece di nasconderli.

11. Perché i test non usano un modello reale? Cosa non misurano?

I 60 test usano `ScriptedModel`, un modello finto che restituisce risposte preparate. Servono a verificare i contratti deterministici: grafo, routing, privacy, HITL, ledger, memoria, policy, RAG e parsing. Non richiedono LM Studio, non usano rete e non dipendono dall'ordine di esecuzione.

Non misurano la qualità del modello, la latenza o la capacità di scegliere il tool. Per quello c'è la suite di valutazione con LFM3 e Llama. Le due cose sono complementari: i test impediscono regressioni nel codice, l'eval misura il comportamento del sistema.

12. Cosa faresti con più tempo o in produzione?

Prima di tutto un insieme di valutazione held-out e più formulazioni per scenario, per distinguere la copertura dalla generalizzazione. Poi un ledger transazionale su database, invece di file JSON con lock di processo, e un audit con retention e accesso controllato.

Sul retrieval, un corpus più grande richiederebbe ricerca semantica o ibrida. Sul modello, servirebbe una misura di Gemini o di un modello più grande per capire quanto margine c'è. Non aggiungerei multi-agente o reflection senza un'ipotesi misurabile che ne giustifichi il costo.

Trabocchetti e domande scomode

Queste domande servono a verificare se il confine è chiaro anche quando il caso non è stato previsto. La risposta migliore dichiara il limite e indica cosa servirebbe.

`last_human_index` restituisce 0 se non trova messaggi umani. È un rischio?

Nel grafo ogni turno parte da `start_turn`, che aggiunge una `HumanMessage`; quindi il caso non si presenta nel flusso normale. Se la funzione fosse chiamata su uno stato senza richiesta umana, la lettura partirebbe dall'inizio e potrebbe vedere una risposta vecchia. È un motivo per non usarla fuori dal contratto del turno.

`pending_interrupt` prende solo il primo interrupt. Perché?

Il grafo attuale apre un solo interrupt per turno: la proposta di rimborso. Prendere il primo è sufficiente e rende la UI prevedibile. Se in futuro un turno potesse aprire più richieste, servirebbe una coda e una gestione esplicita dell'ordine.

Il ledger scrive refunds e orders in due passaggi. Cosa succede se il processo si interrompe?

Ogni file viene sostituito atomicamente, ma la coppia non è una transazione unica. Un crash tra le due scritture può lasciare refunds aggiornato e orders no. Per la demo a processo singolo è accettabile; in produzione servirebbe una transazione su database o un journal di recupero.

`sources_from_messages` interpreta solo JSON. E se un tool restituisse testo?

Il `try` su `json.loads` salta il messaggio e la risposta resta senza fonti. È voluto: solo `cerca_policy` produce fonti, e il suo contratto è JSON con `contesto` e `fonti`. Un tool nuovo che volesse esporre fonti dovrebbe rispettare lo stesso payload.

`dedupe_models` raggruppa per famiglia prima di @. Non perde varianti?

Sì, per costruzione mostra un alias per famiglia: `lfm3-1.7b@q4_k_m` e `lfm3-1.7b` diventano una voce, con il preferito che vince. È una scelta di UI, non di runtime: il modello scelto viene passato per intero a LM Studio. Se servissero più quantizzazioni nello stesso selettore, la chiave andrebbe cambiata.

Perché `final_reasoning_effort` vale solo per LFM3 locale?

È l'unico modello del progetto per cui il controllo `reasoning_effort` è stato verificato. Su Gemini o su altri modelli locali non applico un'opzione che non conosco: il rischio sarebbe cambiare il comportamento senza una misura. La risposta finale usa `none` solo dove serve.

Scaletta della demo (5–10 minuti)

- Apri con la tesi: l'LLM decide, il codice vincola. Non partire dai file.
- Mostra il grafo a quattro nodi e spiega perché `tools` non torna ad `agent`.
- Apri `tools.py` e indica `_customer`: il cliente non è un argomento del modello.
- Esegui «Quanto costa la spedizione?» e mostra le fonti `[S1]`, `[S2]`.
- Esegui il rimborso di ORD-1001: mostra la sospensione, approva e indica il record nel ledger.
- Esegui «Mostrami ORD-1004»: stesso messaggio di un ordine inesistente e audit dell'accesso negato.
- Esegui il prompt injection: il rimborso resta in attesa, non viene eseguito.
- Chiudi con la failure gallery di Llama e con il limite dei 31 scenari.